Recoverability in DBMS

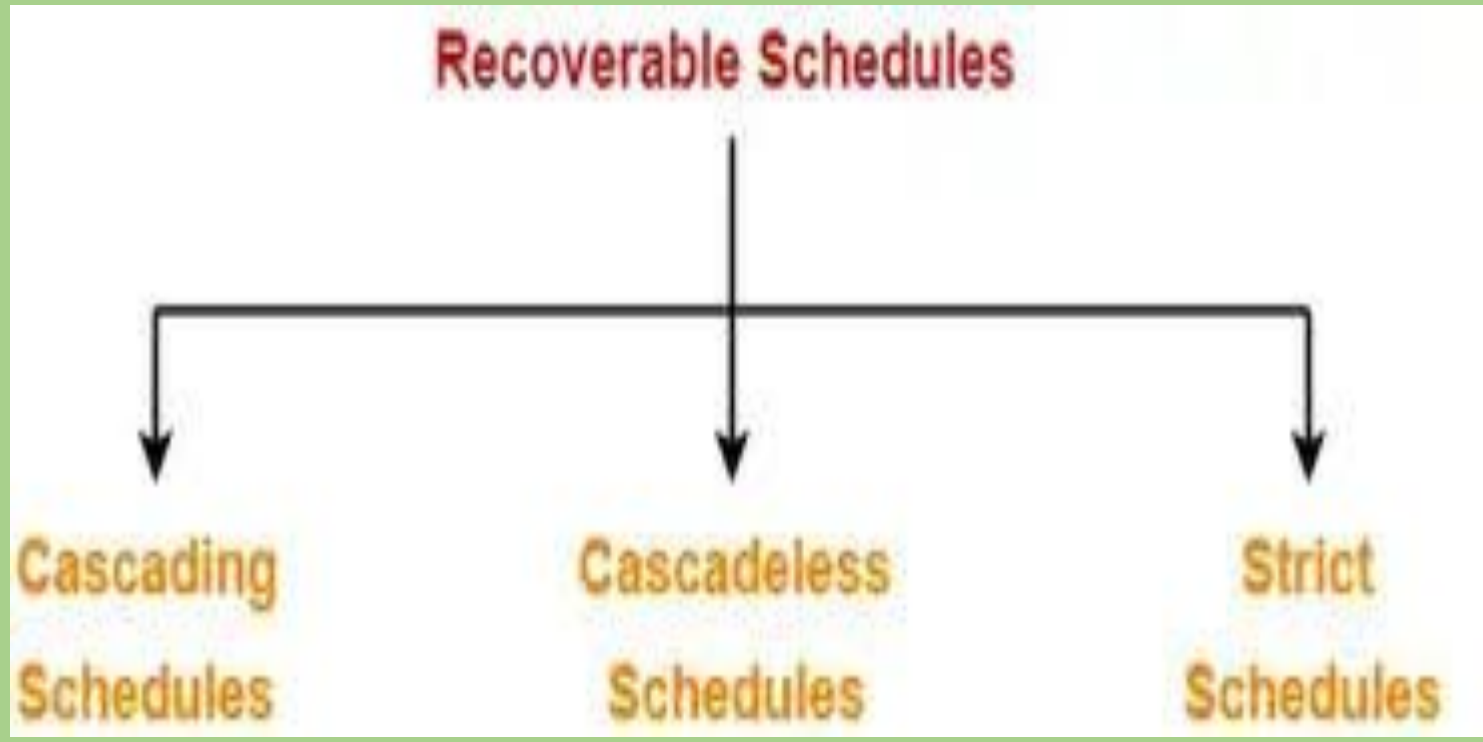
Recoverability

- **Recoverability** is a property of database systems that ensures that, in the event of a failure or error, the system can recover the database to a consistent state.
- Recoverability guarantees that all committed transactions are durable and that their effects are permanently stored in the database, while the effects of uncommitted transactions are undone to maintain data consistency.
- The recoverability property is enforced through the use of transaction logs, which record all changes made to the database during transaction processing. When a failure occurs, the system uses the log to recover the database to a consistent state, which involves either undoing the effects of uncommitted transactions or redoing the effects of committed transactions.

Types of Schedules based Recoverability in DBMS

- Schedules Based on Recoverability

- Recoverable Schedule
- Cascadeless Schedule
- Strict Schedule



Recoverable Schedule:

- A schedule is recoverable if it allows for the recovery of the database to a consistent state after a transaction failure.
- In a recoverable schedule, a transaction that has updated the database must commit before any other transaction reads or writes the same data.
- If a transaction fails before committing, its updates must be rolled back, and any transactions that have read its uncommitted data must also be rolled back.

. Only reads are allowed before write operations on the same data. Only reads (Ti->Tj) are permissible.

Example 1: Consider the following schedule involving two transactions T_1 and T_2.

T_1	T_2
R(A)	
W(A)	
	W(A)
	R(A)
commit	
	commit

This is a recoverable schedule since T1 commits before T2, that makes the value read by T2 correct.

Example 2:

S1: R1(x), W1(x), R2(x), R1(y), R2(y),
W2(x), W1(y), C1, C2;

Given schedule follows order of Ti->Tj => C1->C2. Transaction T1 is executed before T2 hence there is no chances of conflict occur. R1(x) appears before W1(x) and transaction T1 is committed before T2 i.e. completion of first transaction performed first update on data item x, hence given schedule is recoverable.

Cascadeless Schedule:

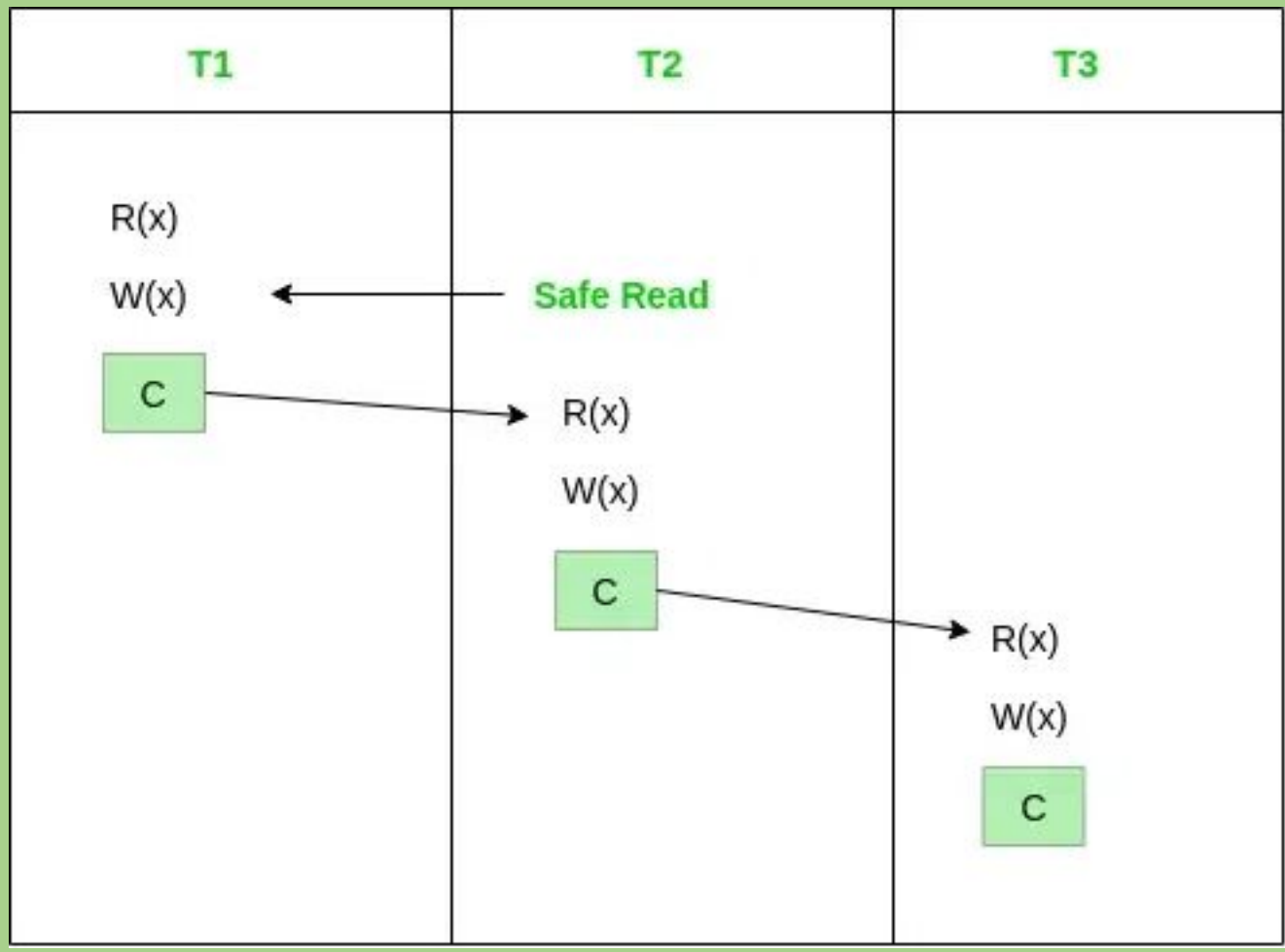
- A schedule is cascaded less if it does not result in a cascading rollback of transactions after a failure.
- In a cascade-less schedule, a transaction that has read uncommitted data from another transaction cannot commit before that transaction commits.
- If a transaction fails before committing, its updates must be rolled back, but any transactions that have read its uncommitted data need not be rolled back.
- When no **read** or **write-write** occurs before the execution of the transaction then the corresponding schedule is called a cascadeless schedule.

- Example:
- S3: R1(x), R2(z), R3(x), R1(z), R2(y), R3(y), W1(x), C1,
- W2(z), W3(y), W2(y), C3, C2;

- In this schedule W3(y) and W2(y) overwrite conflicts and there is no read, therefore given schedule is cascade less schedule.

Cascadeless Example:

- As given below all the transactions
- are reading committed data hence
- it’s cascadeless schedule.



Irrecoverable Schedule:

- The table below shows a schedule with two transactions, T1 reads and writes A and that value is read and written by T2.
- T2 commits. But later on, T1 fails. So we have to rollback T1. Since T2 has read the value written by T1, it should also be rollbacked.
- But we have already committed that. So this schedule is irrecoverable schedule.
- When Tj is reading the value updated by Ti and Tj is committed before committing of Ti, the schedule will be irrecoverable.

Strict Schedule:

- A schedule is strict if it is both recoverable and cascades.
- In a strict schedule, a transaction that has read uncommitted data from another transaction cannot commit before that transaction commits, and a transaction that has updated the database must commit before any other transaction reads or writes the same data. If a transaction fails before committing, its updates must be rolled back, and any transactions that have read its uncommitted data must also be rolled back.

strict schedule

- If the schedule contains no **read** or **write** before commit then it is known as a strict schedule. A strict schedule is strict in nature.
- Example:
- S4: R1(x), R2(x), R1(z), R3(x), R3(y),
- W1(x), C1, W3(y), C3, R2(y), W2(z), W2(y), C2;
- In this schedule, no read-write or write-write conflict arises before committing hence its strict schedule.

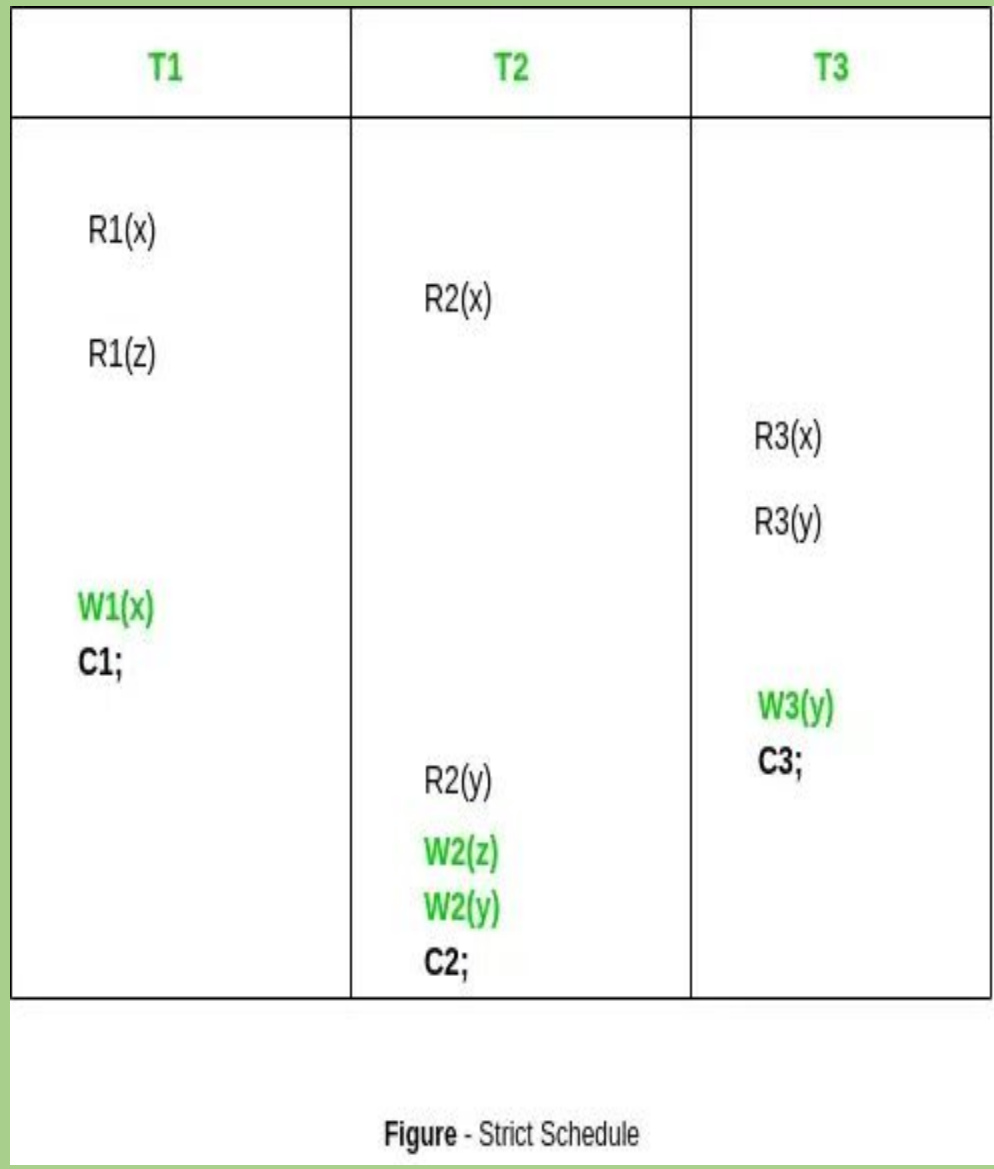


Figure - Strict Schedule

Cascading Abort:

- Cascading Abort can also be rollback. If transaction T1 aborts as T2 read data that is written by T1 it is not committed. Hence its cascading rollback.

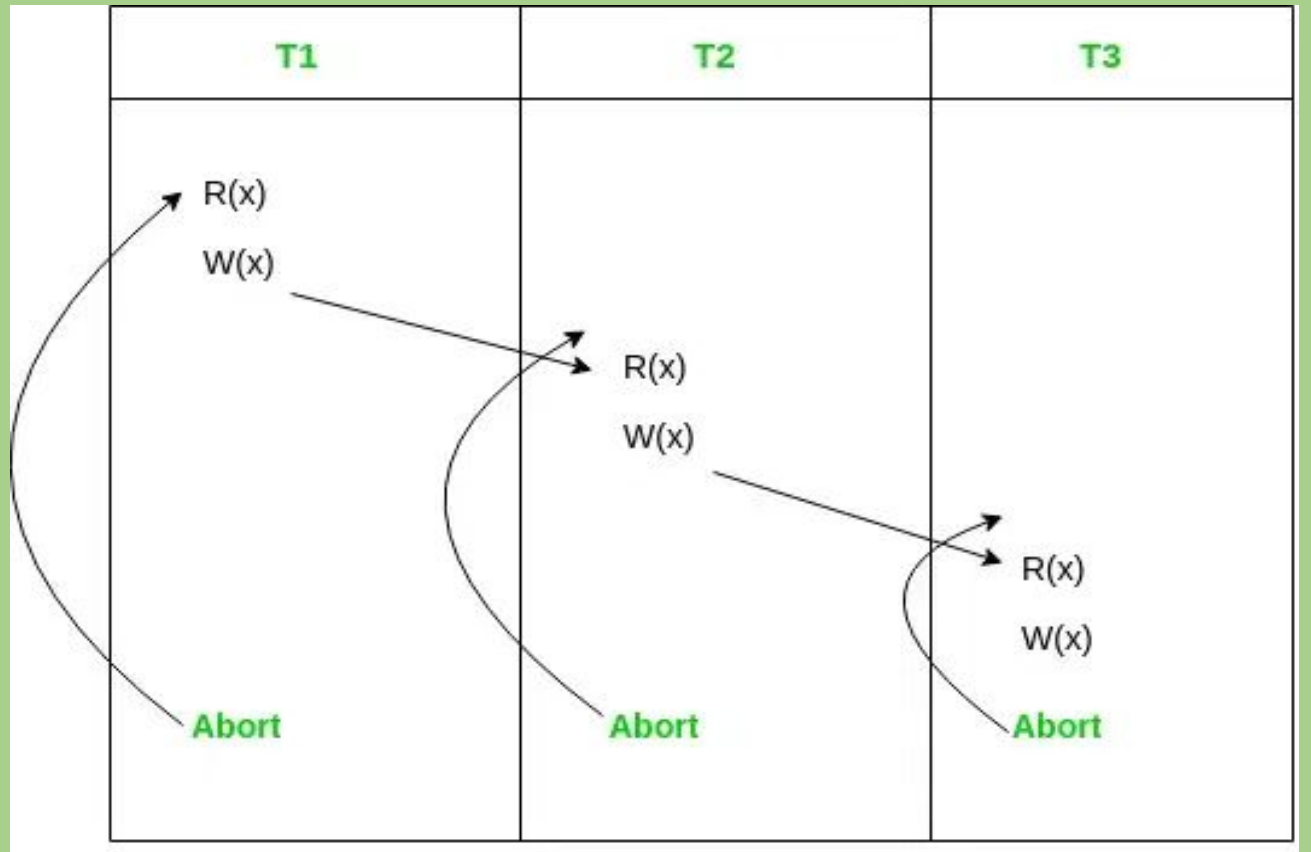


Figure - Cascading Abort

Co-Relation between Strict, Cascadeless, and Recoverable schedules

- Below is the picture showing the correlation between Strict Schedules, Cascadeless Schedules, and Recoverable Schedules.

Figure - Venn diagram of these schedules